

```

verifier et metre commentaire from flask import Blueprint,
jsonify

```

```

from database import db, Participants, Courses
from utils import get_error, get_json_body

```

```

participant_bp = Blueprint("participants", __name__)

```

```

# =====

```

 ChatGPT ▾  Mettre à niveau Afficher plus ▾

 ...

Ton code est globalement correct, mais il y a **un problème important** : la méthode `from_body()` **ne doit pas être dans ce fichier de routes**. Elle appartient au modèle `Participants` (dans `database.py` ou `models.py`). Les routes ne doivent contenir que la logique HTTP.

Voici une version **corrigée et commentée** de ton fichier `routes/participants.py`.

</> Python



▶ Exécuter

```

from flask import Blueprint, jsonify

# Import des modèles et de l'objet SQLAlchemy
from database import db, Participants, Courses

# Import des fonctions utilitaires
from utils import get_error, get_json_body

# Création du Blueprint pour les routes des participants
participant_bp = Blueprint("participants", __name__)

# =====
# CREATE : Ajouter un participant à un cours
# POST /courses/<id>/participants
# =====
@participant_bp.post("/courses/<int:id>/participants")
def create_participant(id):
    """Ajoute un participant dans un cours."""

```

```
# Recherche du cours
course = db.session.get(Courses, id)

# Vérifie que le cours existe
if course is None:
    return get_error("COURSE_NOT_FOUND")

# Vérifie que le cours n'est pas complet
if len(course.participants) >= course.seats:
    return get_error("COURSE_FULL")

# Récupération du corps JSON de la requête
body = get_json_body()

# Création du participant
participant = Participants()

# Validation et remplissage des données
participant.from_body(body)

# Association du participant au cours
participant.course_id = id

# Sauvegarde en base de données
db.session.add(participant)
db.session.commit()

# Retourne le participant créé
return jsonify(participant.to_dict()), 201

# =====
# READ : Récupérer un participant
# GET /participants/<participant_id>
# =====
@participant_bp.get("/participants/<int:participant_id>")
def get_participant(participant_id):
    """Retourne un participant à partir de son identifiant."""

    # Recherche du participant
    participant = db.session.get(Participants, participant_id)

    # Vérifie que le participant existe
    if participant is None:
        return get_error("PARTICIPANT_NOT_FOUND")

    # Retourne les informations du participant
    return jsonify(participant.to_dict()), 200

# =====
# READ : Récupérer un participant d'un cours
```

```
# GET /courses/<course_id>/participants/<part_id>
# =====
@participant_bp.get("/courses/<int:course_id>/participants/<int:part_id>")
def get_participant_for_course(course_id, part_id):
    """Retourne un participant appartenant à un cours."""

    # Recherche du cours
    course = db.session.get(Courses, course_id)

    # Vérifie que le cours existe
    if course is None:
        return get_error("COURSE_NOT_FOUND")

    # Recherche du participant
    participant = db.session.get(Participants, part_id)

    # Vérifie que le participant existe
    if participant is None:
        return get_error("PARTICIPANT_NOT_FOUND")

    # Vérifie que le participant appartient bien au cours
    if participant.course_id != course_id:
        return get_error("PARTICIPANT_NOT_IN_COURSE")

    # Retourne le participant
    return jsonify(participant.to_dict()), 200

# =====
# UPDATE : Modifier un participant
# PUT /participants/<participant_id>
# =====
@participant_bp.put("/participants/<int:participant_id>")
def update_participant(participant_id):
    """Modifie les informations d'un participant."""

    # Recherche du participant
    participant = db.session.get(Participants, participant_id)

    # Vérifie que le participant existe
    if participant is None:
        return get_error("PARTICIPANT_NOT_FOUND")

    # Récupération du corps JSON
    body = get_json_body()

    # Validation et mise à jour des données
    participant.from_body(body)

    # Sauvegarde des modifications
    db.session.commit()
```

```

# Retourne le participant mis à jour
return jsonify(participant.to_dict()), 200

# =====
# DELETE : Supprimer un participant
# DELETE /participants/<participant_id>
# =====
@participant_bp.delete("/participants/<int:participant_id>")
def delete_participant(participant_id):
    """Supprime un participant."""

    # Recherche du participant
    participant = db.session.get(Participants, participant_id)

    # Vérifie que le participant existe
    if participant is None:
        return get_error("PARTICIPANT_NOT_FOUND")

    # Suppression du participant
    db.session.delete(participant)

    # Validation de la suppression
    db.session.commit()

    # Aucun contenu retourné
    return "", 204

```

À déplacer dans database.py (classe Participants)

La méthode suivante ne doit pas être dans le fichier des routes :

</> Python



▶ Exécuter

```

def from_body(self, body):
    """Valide les données reçues et remplit l'objet Participant."""

    # Vérification des champs obligatoires
    if "firstname" not in body or "lastname" not in body or "age" not in body:
        raise ValueError("MISSING_FIELDS")

    firstname = body["firstname"]
    lastname = body["lastname"]
    age = body["age"]

    # Vérification du prénom
    if not isinstance(firstname, str) or firstname.strip() == "":
        raise ValueError("INVALID_PARTICIPANT_FIRSTNAME")

    # Vérification du nom
    if not isinstance(lastname, str) or lastname.strip() == "":

```

```
raise ValueError("INVALID_PARTICIPANT_LASTNAME")

# Vérification de l'âge
if not isinstance(age, int) or age < 18 or age > 120:
    raise ValueError("INVALID_PARTICIPANT_AGE")

self.firstname = firstname.strip()
self.lastname = lastname.strip()
self.age = age
```

Remarques

-  Les commentaires sont clairs et homogènes.
-  Les routes CRUD sont bien organisées.
-  `from_body()` est retirée du fichier des routes.
-  Il est conseillé de limiter l'âge à **18–120 ans** si c'est la règle métier.
-  Pense également à gérer les `ValueError` levées par `from_body()` (par exemple avec un `try/except` ou un gestionnaire d'erreurs Flask) afin de renvoyer les bons codes HTTP au lieu d'une erreur 500.

